Substitute:

$$2160 = 2000 + 4[(i-6)9 + (j-15)]$$
$$160 = 4[(i-6)9 + (j-15)]$$
$$40 = 9(i-6) + (j-15)$$
$$(9\times4) + 4 = 9(i-6) + (j-15)$$

$\therefore i-6 = 4$ & $j-15 = 4$

$\therefore \boxed{i = 10}$ $\boxed{j = 19}$

Therefore, $(\vec{i}, \vec{j}) = (10, 19)$.

So the element is $\Rightarrow A[10][19]$ whose address is 2160.

Column major order:

$$LOC(A[i][j]) = BA + W[(j-L_2) \times N_1 + (i-L_1)]$$

where,

N_1 = no. of rows
$= 17 - 6 + 1$

$\boxed{N_1 = 12}$

Substitute:

$$LOC(A[10][19]) = 2000 + 4[(19-15)12 + (10-6)]$$
$$= 2000 + 4[4\times12 + 4]$$
$$= 2000 + 4[48+4]$$
$$= 2000 + 208$$

$\boxed{LOC(A[10][19]) = 2208}$.

— X —

(5) A two-dimensional array A[6.....17][15.....23] is stored in memory, having base address 2000 and size of each element in array is 4 bytes.

If the element at address 2160 is stored in row-major order, find the indices (i, j) of the element A[i][j] - find the address of the same element if the array were arranged in column-major order.

⟶ Here Array = [6.....17][15.....23]
Base. A. = 2000
Element size = 4 bytes
Given add. = 2160
Stored in row major order. Then find (i, j)

Row-major address formula:

$$LOC(A[i][j]) = BA + w[(i - L_1)n_2 + (j - L_2)]$$

where; BA = Base address
w = Size of each element in bytes
i = Row index of required element
j = Column index of " "
L_1 = Lower bound of row
L_2 = Lower bound of column
n_2 = no. of column in array
$= 23 - 15 + 1$
$= 9$

VISION

③ Update :

: Algorithm :

Step : 1 : Start
Step : 2 : Read the pos & new value.
Step : 3 : Replace A[pos] with the new value.
Step : 4 : Display the updated array.
Step : 5 : Stop.

④ Search :

: Algorithm :

Step : 1 : Start
Step : 2 : Read the element to be searched.
Step : 3 : Compare it with each element of the array.
Step : 4 : If the element is found, display it's pos.
Step : 5 : If the end of the array is searched, display "Element not found".
Step : 6 : Stop.

⑤ Traversal :

: Algorithm :

Step : 1 : Start.
Step : 2 : Set i = 0
Step : 3 : Repeat while i < n
Step : 4 : Display A[i]
Step : 5 : Increment by 1
Step : 6 : Stop.

(4) Describe algorithms of different operations on array.

- An array is a linear data structure that stores elements of the same type. Contiguous memory locations.

- the main operations:

(1) Insertion:

: Algorithm :

Step: 1 : Start
Step: 2 : Read the element & it's position.
Step: 3 : Set i = n-1
Step: 4 : Repeat while i >= pos & shift A[i] to A[i+1].
Step: 5 : Insert the new element at A[pos]
Step: 6 : Increase n by 1.
Step: 7 : Stop.

(2) Deletion :

: Algorithm :

Step: 1 : Start
Step: 2 : Read the pos of the element to be deleted.
Step: 3 : Set i = pos
Step: 4 : Repeat while i < n-1 & shift A[i+1] to A[i].
Step: 5 : Decrease n by 1.
Step: 6 : Stop.

VISION

(3) State the meaning of time Complexity and space compleaity with respect to algorithms. Explain it with simple examples.

→ (1) Time Complexity:

- Time Complexity is the amount of time or no. of operations an algorithm takes to execute as the i/p size increases. It is generally expressed using Big-O notation (O).

- Ex:
If an algorithm checks every element of an array of size n once, it takes n operations.

Therefore, it's time complexity is O(n).

(2) Space Complexity:

- Space Complexity is the amount of memory or extra space required by an algorithm as the i/p size increases.

- Ex:
If an algorithm creates an additional array of size n, it requires n extra memory locations.

Therefore, it's space complexity is O(n).

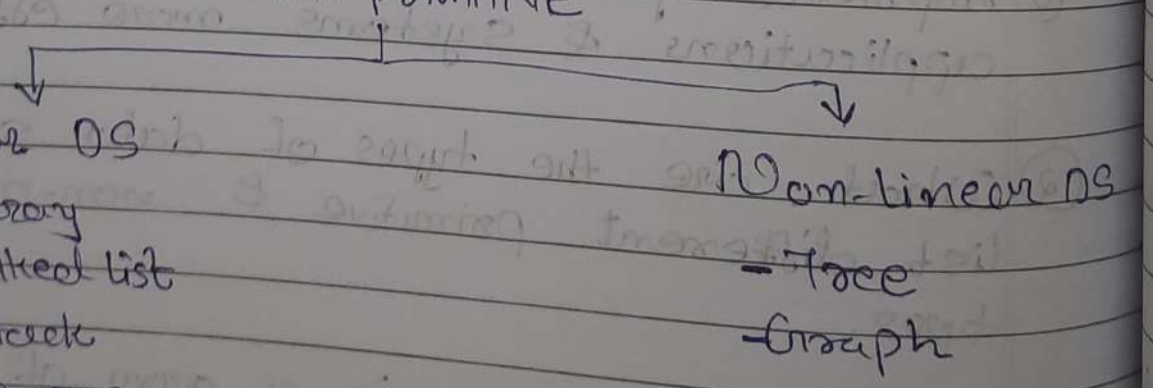
- Data structures are mainly classified into two types:

① Primitive Data structures:

- These are the basic data types provided by C programming language.
- ex:
  - Integer → ex. 10
  - float / double → 10.5
  - Character → 'A'
  - Boolean → true or false.

② Non-Primitive Data structure:

- these are created using primitive data & can store multiple values.
- They are further divided into:



VISION

Assignment - 1

13/8/26

1) Why is data management important in Computer science?

⇒ Data management is important in Computer Science because it helps in storing, organizing, processing & retrieving data efficiently. Good data management ensures that data is accurate, secure, accessible, and available when required.

(1) Efficient storage - Stores large amounts of data systematically.
(2) Easy access - Helps retrieve required data quickly.
(3) Data accuracy - Reduces errors & duplication.
(4) Data security - Protects data from unauthorized access.
(5) Better decision-making - Provides reliable data for analysis & decisions.
(6) Improved performance - makes Computer applications & systems more efficient.

2) What are the types of data structure? list different primitive & non-primitive data types.

⇒ A data structure is a way of organizing and storing data in a computer so that it can be accessed & processed efficiently.